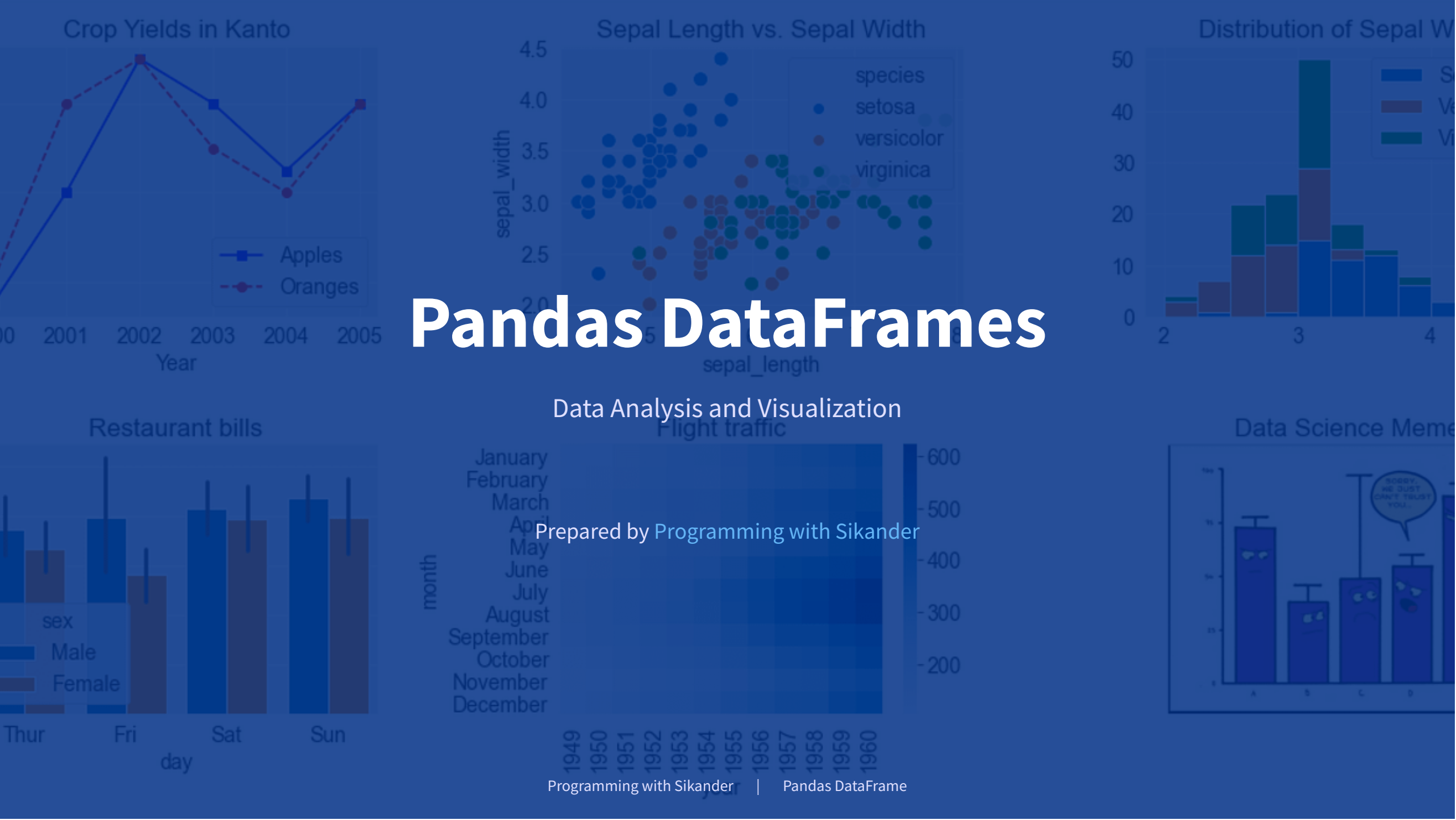




Pandas DataFrames

Data Analysis and Visualization

Prepared by Programming with Sikander

Topics

- 1 DataFrame
- 2 Creating DataFrame from Dictionary
- 3 Selecting specific columns
- 4 Selecting rows using **loc**
- 5 Selecting rows using **iloc**
- 6 Adding and Removing rows and columns
- 7 Filtering DataFrames
- 8 Sorting DataFrames
- 9 Unique/nunique/mode/value_counts
- 10 Nlargest/nsmallest
- 11 Rank
- 12 Concat
- 13 Merge

Datasets in Python

Moving from limitations to powerful data manipulation

2D Numpy Array

- Single data type
- Limited functionality
- No column labels

Pandas DataFrame

- ✓ Multiple data types
- ✓ Rich functionality
- ✓ Built on Numpy

Pandas! - High level data manipulation tool

Created by Wes McKinney

DataFrame

DataFrame is the main object in Pandas, used to represent data with rows and columns (tabular or excel spreadsheets)

	Column 1	Column 2	Column 3	Column 4
Row 1	Data	Data	Data	Data
Row 2	Data	Data	Data	Data
Row 3	Data	Data	Data	Data
Row 4	Data	Data	Data	Data

Each row
represents
an
observation



Each column
represents a
field/variable



Rows have Row Index



Columns have Column
Index

Creating a DataFrame from Dictionary

```
import pandas as pd

# Create a dictionary
data = {
    'Country': ['Brazil', 'Russia', 'India',
                'China', 'South Africa'],
    'Capital': ['Brasília', 'Moscow', 'New
Delhi', 'Beijing', 'Pretoria'],
    'Area': [8.516, 17.10, 3.286, 9.597, 1.221],
    'Population': [200.4, 143.5, 1252, 1357,
52.98]
}

# Create DataFrame from dictionary
df = pd.DataFrame(data)

print(df)
```

Resulting DataFrame

	Country	Capital	Area	Population
0	Brazil	Brasília	8.516	200.4
1	Russia	Moscow	17.10	143.5
2	India	New Delhi	3.286	1252
3	China	Beijing	9.597	1357
4	South Africa	Pretoria	1.221	52.98



Dictionary keys become **column names**



Dictionary values become **column data**



Row index starts from **0 by default**

Head and Tail Functions

↑ Head Function

Returns the **first n rows** of the DataFrame based on position.
Useful for quickly testing if your object has the right type of data.

```
# Default: first 5 rows
df.head()

# Custom: first 3 rows
df.head(3)
```

df.head(3)

	Country	Capital	Area
0	Brazil	Brasília	8.516
1	Russia	Moscow	17.10
2	India	New Delhi	3.286

↓ Tail Function

Returns the **last n rows** of the DataFrame based on position.
Useful for quickly verifying data, especially after sorting or appending.

```
# Default: last 5 rows
df.tail()

# Custom: last 3 rows
df.tail(3)
```

df.tail(3)

	Country	Capital	Area
2	India	New Delhi	3.286
3	China	Beijing	9.597
4	South Africa	Pretoria	1.221

Accessing Columns in DataFrames

↳ Single Brackets []

Returns a **Series** object containing the column data

```
# Returns a Series
df['Country']

# Result type
type(df['Country'])
# pandas.core.series.Series
```

↳ Double Brackets [[]]

Returns a **DataFrame** containing the selected column

```
# Returns a DataFrame
df[['Country']]

# Result type
type(df[['Country']])
# pandas.core.frame.DataFrame
```

↳ Attribute Access

Access column as an **attribute** of the DataFrame

```
# Access as attribute
df.Country

# Same as single brackets
df['Country']
# Both return a Series
```

df['Country'] (Series)

0	Brazil
1	Russia
2	India
3	China
4	South Africa

df[['Country']] (DataFrame)

	Country
0	Brazil
1	Russia
2	India
3	China
4	South Africa

Using .loc and .iloc Accessors

📌 .loc Accessor

Access rows and columns by **label** (index name)

```
# Access a row by label
df.loc['India']

# Access specific cell
df.loc['India', 'Capital']

# Slice by labels
df.loc['Brazil':'China']
```

☰ .iloc Accessor

Access rows and columns by **position** (integer index)

```
# Access a row by position
df.iloc[2]

# Access specific cell
df.iloc[2, 1]

# Slice by positions
df.iloc[0:3]
```

Example DataFrame

	Country	Capital	Area
0	Brazil	Brasília	8.516
1	Russia	Moscow	17.10
2	India	New Delhi	3.286
3	China	Beijing	9.597

Access Examples

df.iloc[2] - Row at position 2

df.loc[2] - Row with index label 2

df.iloc[2, 1] - Cell at row 2, column 1

df.loc[2, 'Capital'] - Cell at row 2, 'Capital' column

df.iloc[1:3] - Rows at positions 1 and 2

df.loc[1:3] - Rows with index labels 1, 2, and 3



.loc uses **labels** and **includes** the end index



.iloc uses **positions** and **excludes** the end index

Programming with Sikander



Pandas DataFrame

Adding and Removing Rows and Columns

+ Adding Rows

Add new rows to a DataFrame using **append()** or **loc**

```
# Using loc
df.loc['new_index'] = ['value1', 'value2', 'value3']

# Using append
new_row = {'col1': 'value1', 'col2': 'value2'}
df = df.append(new_row, ignore_index=True)
```

- Removing Rows

Remove rows using **drop()** method with index labels

```
# Drop single row
df = df.drop('index_label')

# Drop multiple rows
df = df.drop(['index1', 'index2'])

# Modify DataFrame in place
df.drop('index', inplace=True)
```

+ Adding Columns

Add new columns using **direct assignment** or **insert()**

```
# Direct assignment
df['new_column'] = ['value1', 'value2', 'value3']

# Calculated column
df['density'] = df['population'] / df['area']

# Insert at specific position
df.insert(1, 'new_column', values)
```

- Removing Columns

Remove columns using **drop()** with axis=1 or **del**

```
# Using drop with axis=1
df = df.drop('column_name', axis=1)

# Alternative syntax
df = df.drop(columns=['col1', 'col2'])

# Using del keyword
del df['column_name']
```

Before

	Country	Area	Population
0	Brazil	8.516	200.4
1	Russia	17.10	143.5

After Adding Density Column

	Country	Area	Population	Density
0	Brazil	8.516	200.4	23.54
1	Russia	17.10	143.5	8.39



drop() method **returns a new DataFrame** unless inplace=True



Use **del** for immediate removal without creating a copy

Filtering DataFrames

≡ Single Condition

Apply **relational operations** to DataFrame columns to filter data

```
# Filter by area greater than 10
df[df['Area'] > 10]

# Filter by country name
df[df['Country'] == 'India']

# Filter by population range
df[(df['Population'] >= 100) & (df['Population']
<= 500)]
```

▼ Multiple Conditions

Combine conditions using **&** (AND) and **|** (OR) operators

```
# Multiple conditions with AND
df[(df['Area'] > 5) & (df['Population'] > 200)]

# Multiple conditions with OR
df[(df['Country'] == 'India') | (df['Country']
== 'China')]

# Complex conditions
df[((df['Area'] > 10) & (df['Population'] >
200)) | (df['Country'] == 'India')]
```

df[df['Area'] > 10]

	Country	Capital	Area	Population
1	Russia	Moscow	17.10	143.5
3	China	Beijing	9.597	1357

df[(df['Area'] > 5) & (df['Population'] > 200)]

	Country	Capital	Area	Population
3	China	Beijing	9.597	1357

! Use **&** and **|** instead of **and** and **or**

⋮ Wrap each condition in **parentheses**

✓ Filtering returns a **new DataFrame** with matching rows

Sorting DataFrames

☰ Single Column

Sort DataFrame by **one column** in ascending or descending order

```
# Ascending order
df.sort_values(by='Area')

# Descending order
df.sort_values(by='Population', ascending=False)
```

☰ Index Sorting

Sort DataFrame by its **index** rather than column values

```
# Sort by index (ascending)
df.sort_index()

# Sort by index (descending)
df.sort_index(ascending=False)

# Sort by column labels
df.sort_index(axis=1)
```

⌵ Multiple Columns

Sort by **multiple columns** with different sort orders

```
# Sort by multiple columns
df.sort_values(by=['Area', 'Population'])

# Different sort orders
df.sort_values(by=['Area', 'Population'],
               ascending=[True, False])
```

⚙️ Additional Options

Control how missing values are handled and sorting behavior

```
# Handle missing values
df.sort_values(by='Area', na_position='first')

# Sort in place
df.sort_values(by='Area', inplace=True)

# Preserve original index
df.sort_values(by='Area', ignore_index=False)
```

df.sort_values(by='Area')

	Country	Area	Population
4	South Africa	1.221	52.98
2	India	3.286	1252
0	Brazil	8.516	200.4
3	China	9.597	1357
1	Russia	17.10	143.5

df.sort_values(by=['Population', 'Area'], ascending=[False, True])

	Country	Area	Population
3	China	9.597	1357
2	India	3.286	1252
0	Brazil	8.516	200.4
1	Russia	17.10	143.5
4	South Africa	1.221	52.98

Working with Categorical Data

Analyze categorical data with methods that help identify **patterns**, **frequencies**, and **unique values**

unique()

Returns **unique values** in a Series or 1-D array

```
# Get unique values
df['Country'].unique()

# Example output
['Brazil', 'Russia', 'India', 'China', 'South Africa']
```

nunique()

Returns the **count of unique elements** in a Series

```
# Count unique values
df['Country'].nunique()

# Example output
5
```

mode()

Returns the **most frequently occurring** value(s)

```
# Get most frequent value
df['Category'].mode()

# Example output
0 'Electronics'
dtype: object
```

value_counts()

Returns counts of unique values in **descending order**

```
# Count occurrences of each value
df['Category'].value_counts()

# Example output
'Electronics' 45
'Clothing' 32
'Food' 18
```

Sample Data

	Player	Country	Category
0	Rohit	India	Batsman
1	Virat	India	Batsman
2	Steve	Australia	Batsman
3	David	Australia	Bowler
4	Joe	England	Batsman

value_counts() Output

Country:
India 2
Australia 2
England 1

Category:
Batsman 4
Bowler 1

Ranking Data

What is Ranking?

The **rank()** method determines the **relative position** of values in a group of data

```
# Basic ranking
df['Rank'] = df['CGPA'].rank()

# Descending order (highest gets rank 1)
df['Rank'] = df['CGPA'].rank(ascending=False)

# Full syntax
df.rank(axis=0, method='average',
        numeric_only=None, na_option='keep',
        ascending=True, pct=False)
```

Ranking Methods

Different methods to handle **ties** when values are equal

```
'average' - average rank of group
df['CGPA'].rank(method='average')

'min' - lowest rank in group
df['CGPA'].rank(method='min')

'max' - highest rank in group
df['CGPA'].rank(method='max')

'first' - ranks assigned in order
df['CGPA'].rank(method='first')
```

Original Data

	Name	Section	CGPA
0	Bob	A	9.2
1	Charlie	B	8.3
2	Alice	A	9.8
3	David	A	8.1
4	Eve	B	7.5

After Ranking by CGPA (Descending)

	Name	Section	CGPA	Rank
0	Bob	A	9.2	2.0
1	Charlie	B	8.3	3.0
2	Alice	A	9.8	1.0
3	David	A	8.1	4.0
4	Eve	B	7.5	5.0

 **ascending:** False for highest values first

 **pct:** True for percentile ranks

 **na_option:** 'keep', 'top', or 'bottom'

 **axis:** 0 for rows, 1 for columns

Merging DataFrames

⤴ What is Merging?

The **merge()** function combines DataFrames based on **common columns** or index

```
# Basic merge on common column
pd.merge(df1, df2, on='key_column')

# Merge with different column names
pd.merge(df1, df2, left_on='col1',
right_on='col2')

# Merge on index
pd.merge(df1, df2, left_index=True,
right_index=True)
```

⚙ Join Types

Different ways to combine DataFrames based on **how** you want to handle non-matching keys

```
# Inner join (intersection)
pd.merge(df1, df2, how='inner')

# Outer join (union)
pd.merge(df1, df2, how='outer')

# Left join (all from left)
pd.merge(df1, df2, how='left')

# Right join (all from right)
pd.merge(df1, df2, how='right')
```

⌄ Inner Join

Returns only rows with **matching keys** in both DataFrames

⌄ Outer Join

Returns **all rows** from both DataFrames, with NaN where no match

← Left Join

Returns **all rows** from left DataFrame and matching from right

→ Right Join

Returns **all rows** from right DataFrame and matching from left

Left DataFrame

ID	Name	Dept
101	Alice	IT
102	Bob	HR
103	Charlie	Finance

Right DataFrame

ID	Salary	Location
101	75000	New York
102	65000	Boston
104	80000	Chicago

Inner Join Result

ID	Name	Dept	Salary	Location
101	Alice	IT	75000	New York
102	Bob	HR	65000	Boston



Specify **on** parameter for common column name



Use **left_on** and **right_on** for different column names



Control duplicate columns with **suffixes** parameter

Additional Data Cleaning Techniques

dropna()

Remove rows or columns with **missing values**

```
# Drop rows with any missing values
df.dropna()

# Drop columns with any missing values
df.dropna(axis=1)

# Drop rows where all values are missing
df.dropna(how='all')

# Drop rows with missing values in specific columns
df.dropna(subset=['Age', 'Salary'])
```

Duplicate Handling

Identify and remove **duplicate rows**

```
# Check for duplicates
df.duplicated()

# Count duplicates
df.duplicated().sum()

# Remove duplicate rows
df.drop_duplicates()

# Keep first or last occurrence
df.drop_duplicates(keep='first')
df.drop_duplicates(keep='last')
```

Data Type Conversion

Convert columns to **appropriate data types**

```
# Convert to specific type
df['Price'] = df['Price'].astype('float')

# Convert to numeric (handles errors)
pd.to_numeric(df['Age'], errors='coerce')

# Convert to datetime
pd.to_datetime(df['Date'])

# Convert multiple columns
df = df.astype({'ID': 'int', 'Price': 'float'})
```

Original Data with Issues

ID	Name	Price	Date
001	Product A	19.99	2023-01-15
002	Product B	"\$25.50"	2023-01-16
003	Product A	19.99	2023-01-15
004	Product C	NaN	2023-01-18
005	Product D	15.75	"Jan 20, 2023"

After Cleaning

ID	Name	Price	Date
1	Product A	19.99	2023-01-15
2	Product B	25.50	2023-01-16
4	Product C	15.75	2023-01-18
5	Product D	15.75	2023-01-20



Use **inplace=True** to modify DataFrame directly



Handle **errors** in type conversion with 'coerce' or 'ignore'



Chain operations: **df.dropna().drop_duplicates()**

Working with Categorical Data

unique()

Returns **unique values** in a Series or 1-D array

```
# Get unique values
df['Country'].unique()

# Example output
array(['Brazil', 'Russia', 'India', 'China'])
```

nunique()

Returns the **number of unique elements** in a Series

```
# Count unique values
df['Country'].nunique()

# Example output
4
```

mode()

Returns the **most frequently occurring** value(s)

```
# Get mode value
df['Category'].mode()

# Example output
0 'Electronics'
dtype: object
```

value_counts()

Returns counts of **unique values** in **descending order**

```
# Count occurrences of each value
df['Country'].value_counts()

# Example output
India 3
Brazil 2
China 1
```

Sample DataFrame

	Player	Country	Team
0	Virat	India	RCB
1	ABD	South Africa	RCB
2	Maxwell	Australia	RCB
3	Chahal	India	RCB
4	Faf	South Africa	RCB

value_counts() Example

```
df['Country'].value_counts()
```

```
India 2
South Africa 2
Australia 1
```

```
df['Country'].unique()
```

```
['India', 'South Africa', 'Australia']
```

```
df['Country'].nunique()
```

```
3
```

Concatenating and Merging DataFrames

⤴ Concatenating DataFrames

Combine DataFrames along a specified axis (rows or columns) using **pd.concat()**

```
# Concatenate vertically (default)
result = pd.concat([df1, df2])

# Concatenate horizontally
result = pd.concat([df1, df2], axis=1)

# Preserve original index
result = pd.concat([df1, df2],
                    ignore_index=False)
```

Concatenation Example

	Student	Sec	Marks
0	Alice	A	85
1	Bob	A	78
0	Charlie	B	92
1	David	B	88

pd.concat([df_A, df_B])

👤 Merging DataFrames

Perform **database-style join operations** based on common columns or indexes using **merge()**

```
# Inner join (default)
result = pd.merge(df1, df2, on='key')

# Left join
result = pd.merge(df1, df2, on='key',
                  how='left')

# Merge on different column names
result = pd.merge(df1, df2, left_on='id',
                  right_on='student_id')
```

Merge Example

Student	Marks	Age
Alice	85	20
Bob	78	21
Charlie	92	19

pd.merge(df_marks, df_age, on='Student')



concat() stacks DataFrames (adds rows or columns)



merge() combines DataFrames based on common values



Choose **how** parameter: inner, outer, left, right

Merging DataFrames

⤴ What is Merging?

The `merge()` function combines DataFrames based on **common columns** or index

```
# Basic merge on common column
pd.merge(df1, df2, on='key_column')

# Merge with different column names
pd.merge(df1, df2, left_on='col1',
right_on='col2')

# Merge on index
pd.merge(df1, df2, left_index=True,
right_index=True)
```

⚙ Join Types

Different ways to combine DataFrames based on **how** you want to handle non-matching keys

```
# Inner join (intersection)
pd.merge(df1, df2, how='inner')

# Outer join (union)
pd.merge(df1, df2, how='outer')

# Left join (all from left)
pd.merge(df1, df2, how='left')

# Right join (all from right)
pd.merge(df1, df2, how='right')
```

⌄ Inner Join

Returns only rows with **matching keys** in both DataFrames

⌄ Outer Join

Returns **all rows** from both DataFrames, with NaN where no match

← Left Join

Returns **all rows** from left DataFrame and matching from right

→ Right Join

Returns **all rows** from right DataFrame and matching from left

Left DataFrame

ID	Name	Dept
101	Alice	IT
102	Bob	HR
103	Charlie	Finance

Right DataFrame

ID	Salary	Location
101	75000	New York
102	65000	Boston
104	80000	Chicago

Inner Join Result

ID	Name	Dept	Salary	Location
101	Alice	IT	75000	New York
102	Bob	HR	65000	Boston



Specify **on** parameter for common column name



Use **left_on** and **right_on** for different column names



Control duplicate columns with **suffixes** parameter

Data Cleaning: Handling Missing Values

fillna()

Fill missing values with a **specific value** or method

```
# Fill with specific value
df.fillna(0)

# Fill with column mean
df['Age'].fillna(df['Age'].mean())

# Fill with column median
df['Salary'].fillna(df['Salary'].median())
```

ffill()

Forward fill - propagate last valid observation forward

```
# Forward fill missing values
df.ffmpeg()

# Forward fill specific column
df['Temperature'].ffmpeg()

# Alternative syntax
df.fillna(method='ffmpeg')
```

bfill()

Backward fill - propagate next valid observation backward

```
# Backward fill missing values
df.bfill()

# Backward fill specific column
df['Temperature'].bfill()

# Alternative syntax
df.fillna(method='bfill')
```

Original Data

Date	Temperature	Humidity
2023-01-01	25.3	65
2023-01-02	NaN	68
2023-01-03	NaN	NaN
2023-01-04	26.7	70
2023-01-05	27.1	NaN

After fillna(0) and ffill()

Date	Temperature	Humidity
2023-01-01	25.3	65
2023-01-02	25.3	68
2023-01-03	25.3	68
2023-01-04	26.7	70
2023-01-05	27.1	70



Use **inplace=True** to modify DataFrame directly



Specify **limit** parameter to control maximum fill distance



Combine methods: **ffill().bfill()** for complete filling

Additional Data Cleaning Techniques

dropna()

Remove rows or columns with **missing values**

```
# Drop rows with any missing values
df.dropna()

# Drop columns with any missing values
df.dropna(axis=1)

# Drop rows where all values are missing
df.dropna(how='all')

# Drop rows with missing values in specific columns
df.dropna(subset=['Age', 'Salary'])
```

Duplicate Handling

Identify and remove **duplicate rows**

```
# Check for duplicates
df.duplicated()

# Count duplicates
df.duplicated().sum()

# Remove duplicate rows
df.drop_duplicates()

# Keep first or last occurrence
df.drop_duplicates(keep='first')
df.drop_duplicates(keep='last')
```

Data Type Conversion

Convert columns to **appropriate data types**

```
# Convert to specific type
df['Price'] = df['Price'].astype('float')

# Convert to numeric (handles errors)
pd.to_numeric(df['Age'], errors='coerce')

# Convert to datetime
pd.to_datetime(df['Date'])

# Convert multiple columns
df = df.astype({'ID': 'int', 'Price': 'float'})
```

Original Data with Issues

ID	Name	Price	Date
001	Product A	19.99	2023-01-15
002	Product B	"\$25.50"	2023-01-16
003	Product A	19.99	2023-01-15
004	Product C	NaN	2023-01-18
005	Product D	15.75	"Jan 20, 2023"

After Cleaning

ID	Name	Price	Date
1	Product A	19.99	2023-01-15
2	Product B	25.50	2023-01-16
4	Product C	15.75	2023-01-18
5	Product D	15.75	2023-01-20



Use **inplace=True** to modify DataFrame directly



Handle **errors** in type conversion with 'coerce' or 'ignore'



Chain operations: **df.dropna().drop_duplicates()**

Thank You

Prepared by **Programming with Sikander**

Watch the full video tutorial



youtube.com/watch?v=5BX0Ktg_drc

